VOL. 20

C++

From the first build to classes, templates and the standard library

Source	cplusplus.com/doc/tutorial
Generated	2026-10-10
Level	Intermediate
Estimated study time	35 hours
Language	English
Volume	Vol. 20 - C++

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume . . . 3
Chapter map . . . 4
Chapter 1 - Toolchain and the structure of a program . . . 4
1.1 Summary . . . 4
1.2 Concepts . . . 5
What the toolchain does . . . 5
The shape of a program . . . 5
Comments . . . 6
Headers, declarations and include guards . . . 6
The standard namespace . . . 7
1.3 Examples . . . 7
1.4 Common mistakes and best practices . . . 9
1.5 Practice exercises . . . 10
Solutions . . . 10
1.6 Mini test . . . 12
Answer key and explanations . . . 13
Chapter 2 - Variables, types and constants . . . 13
2.1 Summary . . . 13
2.2 Concepts . . . 13
The fundamental types . . . 13
Identifiers, declaration and initialisation . . . 14
Constants . . . 15
Unsigned arithmetic and wrapping . . . 15
2.3 Examples . . . 16
2.4 Common mistakes and best practices . . . 18
2.5 Practice exercises . . . 19
Solutions . . . 19
2.6 Mini test . . . 21
Answer key and explanations . . . 22

How to use this volume

Prerequisites. You can edit and save text files on your computer and you have run a command in a terminal before. No previous C or C++ is assumed: the volume starts with the build itself and builds up to classes, templates and the standard library. A little knowledge of another language helps, because ideas such as variables, loops and functions are reused, but C++ spelling differs almost everywhere.

Estimated study time. About 35 hours: two to three hours per chapter, plus the time you spend breaking and fixing your own experiments. Chapters 1 to 7 are the core of the language; chapters 8 and 9 make memory and compound data clear; chapters 10 to 14 are the object oriented library and modern style. Read the examples by typing them, not by skimming: the compiler is the real teacher.

How the chapters work. Each chapter opens with a summary of the ideas, then explains the concepts with tables and diagrams, then shows numbered examples with the exact output they produce, then lists the mistakes that cost the most time together with the best practice that avoids them, and closes with practice exercises, their solutions, and a mini test with a full answer key.

How to run the code. Every block tagged `cpp` is a complete program that compiles as one file. Blocks tagged `text` show the output of the program above them; blocks tagged `bash` or `console` are terminal commands. All C++ examples in this volume were compiled and run with the Microsoft Visual C++ toolchain, version 14.51, using `cl /std:c++17 /EHsc`. Any C++17 compiler produces the same results; where the result depends on the platform, the note under the example says so.

shell

```shell
# Windows, MSVC, from a Developer command prompt
cl /nologo /std:c++17 /EHsc hello.cpp
hello

# Linux or macOS, GCC or Clang
g++ -std=c++17 -Wall -Wextra -o hello hello.cpp
./hello
```

shell

```shell
hello.cpp
Hello, C++!
```

The line `hello.cpp` is the compiler naming the file it is translating; with `/nologo` the banner is suppressed. A warning-free build is not guaranteed by `/W4` alone, so keep `-Wall -Wextra` on GCC and Clang and read the messages.

About the source. The syllabus follows the official C++ tutorial at cplusplus.com, from the introduction to the standard library. The structure, topics and the order in which the language ideas arrive are taken from that tutorial; every sentence, example, exercise and answer in this file was written from scratch for this volume. Modern additions that the tutorial does not cover (constexpr functions, `auto`, range-based `for`, smart pointers, structured bindings) are marked as modern notes and are described from the standard library documentation instead.

Chapter map

#	Chapter	You will learn
1	Toolchain and the structure of a program	what a compiler does, the build steps, `main`, headers and comments
2	Variables, types and constants	fundamental types, initialisation, `const`, `constexpr`, `auto`
3	Operators	arithmetic, comparison, logic, bit operations, precedence
4	Console input and output	`cin` and `cout`, `getline`, formatting with `<iomanip>`
5	Control flow	`if`, `switch`, `while`, `do`, `for`, range-based `for`, jumps
6	Functions, overloads and prototypes	parameters, defaults, scope, recursion, pass by reference
7	Function templates and namespaces	generic functions, name lookup, `using`, the `std` namespace
8	Arrays, character sequences and structs	C arrays, null-terminated strings, `struct`, `enum class`
9	Pointers, references and dynamic memory	`new` and `delete`, pointer arithmetic, smart pointers
10	Classes: encapsulation and constructors	access specifiers, member functions, `this`, static members
11	Special members and operator overloading	copy and move semantics, `operator+`, `operator<<`, conversions
12	Inheritance and polymorphism	base and derived classes, `virtual`, `override`, abstract classes
13	The standard library	`string`, `vector`, `map`, algorithms, file streams
14	Exceptions, casts and a capstone project	`try` and `catch`, the four casts, class templates, a project
-	Cheat sheet	compact reference tables for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - Toolchain and the structure of a program

1.1 Summary

- C++ is a compiled language: a toolchain turns source text into an executable before anything runs.

- The build has three stages: the preprocessor pastes headers in, the compiler turns each translation unit into an object file, the linker joins the objects into one program.
- Every program starts at the `main` function, and the value returned from `main` becomes the exit status of the process.
- A translation unit is one source file together with all the headers it includes.
- Header files hold declarations; include guards stop the same declarations from being pasted twice.
- Comments come in two shapes: `//` to the end of the line, and `/* ... */` which can span lines.
- The standard library is reached through headers such as `<iostream>`, and almost everything in it lives in the `std` namespace.
- Names are case sensitive, and the language has a fixed set of keywords that cannot be used as identifiers.
- The same source compiles on many systems, but the compiler, the flags and the standard library differ between them.

1.2 Concepts

What the toolchain does

A C++ program is never interpreted line by line. Three tools run in sequence, and each one has a narrow job:

Stage	Tool	What happens	Output
Preprocess	preprocessor	includes are replaced by the text of the header, macros are expanded	one translation unit, still text
Compile	compiler	each translation unit is parsed, checked and translated	an object file with machine code
Link	linker	objects and library code are combined, references are resolved	an executable file

The same source file can be compiled twice into two object files, then linked into one program; that is why a project with many files builds faster on the second run: only the changed translation units are compiled again.

The shape of a program

C++

```cpp
#include <iostream>   // a header: declarations from the standard library

int main() {          // the entry point, called by the system
    std::cout << "Hello, C++!";   // a statement, ends with a semicolon
    return 0;                     // exit status: 0 means success
}
```

Reading it line by line:

- `#include <iostream>` is a directive for the preprocessor; it copies the declarations of the input and output streams into the translation unit.
- `int main()` is the function the system calls; a program has exactly one.
- Curly braces group the statements that form the body of the function.
- `std::cout` is the standard output stream; `<<` sends data to it.
- Statements end with a semicolon, and indentation is for humans only.

Comments

C++

```cpp
// A single line comment. Everything up to the end of the line is ignored.

/*
   A block comment. It can span
   as many lines as needed.
*/

int main() {
    int value = 42;      // a comment can follow code on the same line
    return 0;
}
```

The compiler strips comments before parsing, so a comment can hold anything except the end marker of its own form. Use comments to record why a decision was made, not to restate what the code already says.

Headers, declarations and include guards

A large program is split into headers (declarations, usually `.hpp` or `.h`) and source files (definitions, usually `.cpp`). A header tells the compiler what exists; a source file provides the code. When the same header is included twice in one translation unit, declarations would be repeated and the build fails, so headers are wrapped in a guard:

C++

```cpp
#ifndef RECTANGLE_HPP
#define RECTANGLE_HPP

namespace geometry {

class Rectangle {
  public:
    Rectangle(double width, double height);
    double area() const;

  private:
    double width_;
    double height_;
};

}  // namespace geometry

#endif  // RECTANGLE_HPP
```

Modern compilers also accept `#pragma once`, but the macro guard is the portable form and it works everywhere.

The standard namespace

Almost every name in the standard library lives in `namespace std`. The qualifier `std::` says which namespace to look in, so two libraries can use the same name for different things:

C++

```cpp
#include <iostream>

int main() {
    std::cout << "qualified\n";
}
```

Two wider forms exist and both are fine for small programs:

C++

```cpp
using namespace std;    // brings every name of std into scope: convenient, risky

using std::cout;        // brings one name in: the safer habit
```

1.3 Examples

Example 1: the smallest complete program.

C++

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, C++!" << std::endl;
    return 0;
}
```

output

```
Hello, C++!
```

`std::endl` writes a newline and flushes the stream. A single `\n` character is usually enough, because the stream is flushed when the program ends.

Example 2: the build and the run, as the terminal shows them.

shell

```
cl /nologo /std:c++17 /EHsc hello.cpp
hello
```

shell

```
hello.cpp
Hello, C++!
```

The first line is the compiler echo of the file it is translating; the second line comes from the program itself. Change the file and run the same two commands again.

Example 3: several functions in one translation unit.

C++

```cpp
#include <iostream>

int square(int x) {
    return x * x;
}

int cube(int x) {
    return x * square(x);
}

int main() {
    for (int i = 1; i <= 4; ++i) {
        std::cout << i << " squared is " << square(i)
                  << ", cubed is " << cube(i) << '\n';
    }
    return 0;
}
```

output

```
1 squared is 1, cubed is 1
2 squared is 4, cubed is 8
3 squared is 9, cubed is 27
4 squared is 16, cubed is 64
```

`main` sees `square` and `cube` because their definitions appear before it. When definitions come after the caller, a declaration called a prototype is needed.

Example 4: the value returned from `main` is the exit status.

C++

```cpp
#include <iostream>

int main(int argc, char* argv[]) {
    if (argc < 2) {
        std::cout << "usage: greet <name>" << '\n';
        return 1;
    }
    std::cout << "Hello, " << argv[1] << '!' << '\n';
    return 0;
}
```

output

```
usage: greet <name>
```

shell

```shell
greet Ada
echo %ERRORLEVEL%
```

output

```
Hello, Ada!
0
```

`argc` counts the arguments and `argv` holds them, with `argv[0]` being the program name. Run the same program with one argument and it greets by name; run it with none and it returns 1, which shells and scripts read as failure.

Example 5: two translation units and a header.

C++

```cpp
// math_utils.hpp - declarations shared between translation units
#ifndef MATH_UTILS_HPP
#define MATH_UTILS_HPP

namespace math {

int square(int x);
int cube(int x);

}  // namespace math

#endif  // MATH_UTILS_HPP
```

C++

```cpp
// main.cpp - includes the header and defines main
#include <iostream>

#include "math_utils.hpp"

int main() {
    std::cout << math::square(5) << '\n';
    std::cout << math::cube(5) << '\n';
    return 0;
}
```

C++

```cpp
// math_utils.cpp - the definitions
#include "math_utils.hpp"

namespace math {

int square(int x) { return x * x; }

int cube(int x) { return x * square(x); }

}  // namespace math
```

output

```
25
125
```

Angle brackets `<>` search the standard and system include paths, while quotes `""` search the folder of the current file first. Compile all three files together and the linker resolves the calls: `cl /std:c++17 math_utils.cpp main.cpp` or `g++ -std=c++17 math_utils.cpp main.cpp`.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Compiling a file and never linking it	`undefined reference` or `unresolved external symbol` errors appear for every function	build all translation units of the program at once, or use a build tool

Mistake	Why it hurts	Best practice
Forgetting the semicolon after a statement	the compiler reports an error on a later line and the message makes no sense	read the error line and the one above it
Using a keyword as a name, such as `class` or `new`	the file does not compile at all	keep to the keyword list and camelCase or snake_case for identifiers
Copying declarations into two headers without guards	the same declaration appears twice in one translation unit	wrap every header in `#ifndef NAME / #define NAME / #endif`
Assuming `main` can return nothing	a missing return value from `main` is allowed only for `main`, and it hides intent	write `return 0;` explicitly
Comparing output across compilers	type sizes, garbage values and optimisation differ	record the toolchain with the output, as this volume does

1.5 Practice exercises

1. Write a program that prints your name, your city and one line about what you want to build with C++, each on its own line, and returns 0.
2. Extend the program from exercise 1 so that it takes the city as a command line argument and prints a message when the argument is missing.
3. Write a header `stats.hpp` with a guard and two declarations, `average` and `maximum`, for a small array of `double`; then write a `stats.cpp` with the definitions and a `main.cpp` that prints both results for the values 3.5, 9.25 and 7.0.
4. Compile the program from exercise 3 twice in a row and time both builds; then change only `main.cpp`, rebuild, and explain which files the compiler had to translate again and why.

Solutions

1. The program below uses `std::cout` with `\n` and returns an explicit 0.

C++

```cpp
#include <iostream>

int main() {
    std::cout << "Ada\n";
    std::cout << "London\n";
    std::cout << "A small tool that scores board games\n";
    return 0;
}
```

output

```
Ada
London
A small tool that scores board games
```

2. The same program reads `argv[1]` when it is present and falls back to a message plus a return of 1.

C++

```cpp
#include <iostream>

int main(int argc, char* argv[]) {
    if (argc < 2) {
        std::cout << "no city given" << '\n';
        return 1;
    }
    std::cout << "City: " << argv[1] << '\n';
    return 0;
}
```

output

```
no city given
```

Running `city London` prints `City: London` and returns 0.

3. The header holds declarations only, the source holds definitions, and `main` includes the header.

C++

```cpp
// stats.hpp
#ifndef STATS_HPP
#define STATS_HPP

namespace stats {

double average(const double* values, int count);
double maximum(const double* values, int count);

}  // namespace stats

#endif  // STATS_HPP
```

C++

```cpp
// stats.cpp
#include "stats.hpp"

namespace stats {

double average(const double* values, int count) {
    double total = 0.0;
    for (int i = 0; i < count; ++i) {
        total += values[i];
    }
    return total / count;
}

double maximum(const double* values, int count) {
    double best = values[0];
    for (int i = 1; i < count; ++i) {
        if (values[i] > best) {
            best = values[i];
        }
    }
    return best;
}

}  // namespace stats
```

C++

```cpp
// main.cpp
#include <iostream>

#include "stats.hpp"

int main() {
    const double samples[3] = {3.5, 9.25, 7.0};
    std::cout << stats::average(samples, 3) << '\n';
    std::cout << stats::maximum(samples, 3) << '\n';
    return 0;
}
```

output

```
6.58333
9.25
```

4. The first build compiles `main.cpp` and `stats.cpp` and links them. The

second build of an unchanged program is usually faster because the compiler can skip work it has already done only if a build tool records the result; with a plain `cl` command both files are compiled again. Editing `main.cpp` forces that one file to be translated again, while `stats.cpp` is needed by the linker but does not need to be recompiled unless its own source or header changed.

1.6 Mini test

1. Which stage turns source text into machine code? A) the linker B) the compiler C) the preprocessor D) the debugger
2. Which function does every hosted C++ program start from? A) `start` B) `run` C) `main` D) `init`
3. True or false: a translation unit is one source file plus every header it includes.

4. What is the exit status of a program whose `main` returns 0?
5. Which pair of markers surrounds a block comment? A) `//` and `//` B) `/*` and `*/` C) `<!--` and `-->` D) `--[[` and `]]`
6. Why does a header use an include guard?

Answer key and explanations

1. B - the compiler translates each translation unit into an object file; the linker only joins objects.
2. C - the runtime calls `main`, and the program ends when `main` returns.
3. True - the preprocessor pastes the headers into the same translation unit before the compiler runs.
4. Success - 0 is the conventional status for a normal end, and non-zero values report an error.
5. B - `/*` opens a block comment and `*/` closes it; `//` comments out the rest of a line.
6. So that including the same header twice in one translation unit does not repeat the declarations and break the build.

Chapter 2 - Variables, types and constants

2.1 Summary

- A variable is a named region of memory with a type that decides how the bytes are read.
- Fundamental types cover integers, floating point values, characters, booleans and `void`.
- `sizeof` reports the size of a type in bytes, and that size is platform dependent except for the minimum ranges the standard guarantees.
- Signed types hold positive and negative values; unsigned types hold values from zero up to a wider positive maximum.
- A declaration introduces a name and a type; a definition also creates the object; initialisation gives it a first value.
- Brace initialisation refuses narrowing conversions that lose information, which makes it the safest default.
- `const` marks an object that cannot be modified after initialisation, and `constexpr` asks for a value known at compile time.
- Literals have types too: an integer literal is `int`, a floating point literal is `double`, and suffixes such as `u` or `f` change that.
- `auto` lets the compiler deduce the type from the initialiser, which removes repetition and keeps the type correct after a refactor.

2.2 Concepts

The fundamental types

Type	Usual size	Meaning	Example literal
`bool`	1 byte	true or false	`true`
`char`	1 byte	one character, signed or unsigned by platform	`'a'`
`signed char`, `unsigned char`	1 byte	small numbers	`200`
`short`	2 bytes	small integer	`-40`
`unsigned short`	2 bytes	small non-negative integer	`40000`
`int`	4 bytes	the default integer	`42`
`unsigned int`, `unsigned`	4 bytes	non-negative integer	`42u`
`long`	4 or 8 bytes	long integer	`7L`
`long long`	8 bytes	very long integer	`7LL`
`float`	4 bytes	single precision real	`1.5f`
`double`	4 or 8 bytes, usually 8	double precision real	`1.5`
`long double`	8 or more bytes	extended precision real	`1.5L`
`wchar_t`, `char16_t`, `char32_t`	platform	wide characters for Unicode	`L'a'`

Sizes above are the ones of the toolchain used for this volume. Only the guaranteed ranges are portable, which is why portable code asks the library:

C++

```cpp
#include <iostream>
#include <limits>

int main() {
    std::cout << std::numeric_limits<int>::max() << '\n';
    std::cout << std::numeric_limits<int>::min() << '\n';
    std::cout << std::numeric_limits<int>::digits10 << '\n';
}
```

output

```
2147483647
-2147483648
9
```

`digits10` counts digits that survive a round trip through a decimal string, so it is the honest answer when a user asks how many digits an `int` can hold.

Identifiers, declaration and initialisation

An identifier starts with a letter or an underscore and continues with letters, digits and underscores; names are case sensitive, so `total` and `Total` are two different variables. A

declaration is a statement:

C++

```cpp
int count;          // declaration without initialiser: the value is indeterminate
int copies = 3;     // copy initialisation
int items(5);       // direct initialisation
int rows{9};        // brace initialisation: the preferred form
int zero = {};      // value initialisation: zero for arithmetic types
```

Brace initialisation rejects a conversion that loses information:

C++

```cpp
int n{2.5};        // error: narrowing conversion from double to int
int m{300};        // fine on a system with a 4 byte short, error on a 1 byte one
int k{7.0};        // fine: the value is exactly representable, so no data is lost
```

Constants

Form	When the value is known	Can be modified	Typical use
`const` object	run time	no	a value fixed after construction
`constexpr` object	compile time	no	sizes, table constants, unit conversions
`constexpr` function	compile time when arguments are constant	no	compile time computation
`enum`	compile time	no	a closed set of named values
`#define`	preprocessor, text replacement	no	include guards and rare macros

C++

```cpp
const double kUnitPrice = 9.99;              // fixed after initialisation
constexpr int kMonthsPerYear = 12;           // known while compiling
constexpr int square(int x) { return x * x; }

static_assert(kMonthsPerYear == 12, "calendar changed");
```

`static_assert` checks a condition during compilation and stops the build with the message when the condition is false.

Unsigned arithmetic and wrapping

Unsigned arithmetic is defined to wrap: `0u - 1u` is the largest value of the type. Signed overflow has no defined result at all, so it is never used to compute on purpose.

C++

```cpp
unsigned char level = 250;
level = static_cast<unsigned char>(level + 6);   // 256 wraps to 0
```

output

```
level = 0
```

The cast is written out because arithmetic on `unsigned char` promotes to `int` and then converts back; saying it in code makes the intent visible.

2.3 Examples

Example 1: sizes of the fundamental types on one toolchain.

C++

```cpp
#include <iostream>

int main() {
    std::cout << "bool          " << sizeof(bool) << '\n';
    std::cout << "char          " << sizeof(char) << '\n';
    std::cout << "short         " << sizeof(short) << '\n';
    std::cout << "int           " << sizeof(int) << '\n';
    std::cout << "long          " << sizeof(long) << '\n';
    std::cout << "long long     " << sizeof(long long) << '\n';
    std::cout << "float         " << sizeof(float) << '\n';
    std::cout << "double        " << sizeof(double) << '\n';
    return 0;
}
```

output

```
bool          1
char          1
short         2
int           4
long          4
long long     8
float         4
double        8
```

The output comes from 64-bit Windows, where `long` is 4 bytes. On 64-bit Linux and macOS `long` is 8 bytes, so only `sizeof` and `std::numeric_limits` are portable answers.

Example 2: the four initialisation forms.

C++

```cpp
#include <iostream>

int main() {
    int copy = 5;
    int direct(7);
    int braces{9};
    int list = {3};
    int zero{};

    std::cout << copy << ' ' << direct << ' ' << braces << ' '
              << list << ' ' << zero << '\n';

    double ratio = 1.5;
    int narrowed = static_cast<int>(ratio);
    std::cout << "ratio " << ratio << " narrowed " << narrowed << '\n';
    return 0;
}
```

output

```
5 7 9 3 0
ratio 1.5 narrowed 1
```

`static_cast` is the visible way to narrow; it tells the reader that a conversion was meant and lets the compiler check that the types are related.

Example 3: signed and unsigned behaviour.

C++

```cpp
#include <iostream>

int main() {
    unsigned int counter = 0;
    std::cout << "0u - 1u      = " << counter - 1u << '\n';

    unsigned char level = 250;
    for (int i = 0; i < 6; ++i) {
        level = static_cast<unsigned char>(level + 1);
    }
    std::cout << "250 + 6      = " << static_cast<int>(level) << '\n';

    int value = -7;
    std::cout << "-7 / 2       = " << value / 2 << '\n';
    std::cout << "-7 % 2       = " << value % 2 << '\n';
    return 0;
}
```

output

```
0u - 1u      = 4294967295
250 + 6      = 0
-7 / 2       = -3
-7 % 2       = -1
```

Integer division truncates towards zero, so `-7 / 2` is `-3` and the remainder keeps the sign of the dividend. Mixed signed and unsigned comparisons are a different story and are covered in chapter 3.

Example 4: constants and literals.

C++

```cpp
#include <iomanip>
#include <iostream>

const double kPi = 3.141592653589793;
constexpr int kMonthsPerYear = 12;
constexpr int square(int x) { return x * x; }

int main() {
    std::cout << std::setprecision(12);
    std::cout << "pi          = " << kPi << '\n';
    std::cout << "months      = " << kMonthsPerYear << '\n';
    std::cout << "square(9)   = " << square(9) << '\n';
    std::cout << "1.0e3       = " << 1.0e3 << '\n';
    std::cout << "0xFF        = " << 0xFF << '\n';
    std::cout << "sizeof(1L)  = " << sizeof(1L) << '\n';
    std::cout << "sizeof(1.0f)= " << sizeof(1.0f) << '\n';
    return 0;
}
```

output

```
pi          = 3.14159265359
months      = 12
square(9)   = 81
1.0e3       = 1000
0xFF        = 255
sizeof(1L)  = 4
sizeof(1.0f)= 4
```

`0xFF` is hexadecimal for 255, `1.0e3` is a double in scientific notation, and the `L` suffix makes the literal a `long`. Suffixes such as `u`, `l`, `ll` and `f` change the type of the literal, not its value.

Example 5: type deduction with `auto`.

C++

```cpp
#include <iostream>
#include <string>
#include <vector>

int main() {
    auto count = 0;
    auto ratio = 1.5;
    auto label = std::string("volume");
    auto values = std::vector<int>{1, 2, 3};

    std::cout << "count = " << count << '\n';
    std::cout << "ratio = " << ratio << '\n';
    std::cout << "label = " << label << '\n';

    for (auto value : values) {
        std::cout << "value = " << value << '\n';
    }

    const auto& first = values[0];
    std::cout << "first = " << first << '\n';
    return 0;
}
```

output

```
count = 0
ratio = 1.5
label = volume
value = 1
value = 2
value = 3
first = 1
```

`auto` copies the type of the initialiser, so `ratio` is a `double` and `label` is a `std::string`. `const auto&` binds a reference that cannot be used to modify the object and avoids a copy; that form is the one used for the elements of a container in the algorithms of chapter 13.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Reading a variable that was never initialised	the value is indeterminate and the program behaves differently on each run	initialise at the point of declaration, preferably with `{}`
Using a signed value where an unsigned one is expected, such as `vec.size() - 1` on an empty vector	the subtraction wraps to a huge number and the loop runs away	compare with the signed counterpart, or use iterators and `size_t` carefully
Writing `int n{2.5};` and being surprised by the error	brace initialisation forbids narrowing by design	write `static_cast<int>` when the loss of data is intended
Mixing `float` and `double` in one calculation	the result is promoted to `double`, so printed digits appear out of nowhere	pick one type for a value and keep it
Using `#define` for constants	the name disappears from the debugger and obeys no type rules	use `constexpr` or `const`
Wanting a type that fits on every machine	`long` is 4 bytes on Windows and 8 bytes on Linux	ask the standard library with `sizeof` and `std::numeric_limits`

2.5 Practice exercises

1. Write a program that prints `sizeof` for `bool`, `char`, `short`, `int`, `long`, `long long`, `float`, `double` and `long double`, each on its own line with the type name aligned.
2. Write a program that declares `int count = 10;` then prints `count++`, `++count` and the value of `count` after each, and explain the difference in the answer key.
3. Write a program with `constexpr int kFeetPerYard = 3;` and a `constexpr` function `yards(int feet)` that returns whole yards, then prints the result for 10 feet and for 3 feet.
4. Write a program that reads two integers with `std::cin` and prints their sum, their difference, their product and their integer quotient and remainder.

Solutions

1. `sizeof` returns `size_t`, so the stream prints an integer; the type names are written by hand so the columns line up.

C++

```cpp
#include <iostream>

int main() {
    std::cout << "bool        " << sizeof(bool) << '\n';
    std::cout << "char        " << sizeof(char) << '\n';
    std::cout << "short       " << sizeof(short) << '\n';
    std::cout << "int         " << sizeof(int) << '\n';
    std::cout << "long        " << sizeof(long) << '\n';
    std::cout << "long long   " << sizeof(long long) << '\n';
    std::cout << "float       " << sizeof(float) << '\n';
    std::cout << "double      " << sizeof(double) << '\n';
    std::cout << "long double " << sizeof(long double) << '\n';
    return 0;
}
```

output

```
bool        1
char        1
short       2
int         4
long        4
long long   8
float       4
double      8
long double 8
```

2. `count++` is the postfix form: it yields the old value and then increments.

`++count` increments first and yields the new value. Both leave `count` changed by exactly one.

C++

```cpp
#include <iostream>

int main() {
    int count = 10;
    std::cout << "postfix returns " << count++ << '\n';
    std::cout << "count is now   " << count << '\n';
    std::cout << "prefix returns " << ++count << '\n';
    std::cout << "count is now   " << count << '\n';
    return 0;
}
```

output

```
postfix returns 10
count is now   11
prefix returns 12
count is now   12
```

3. A `constexpr` function can be evaluated while compiling, so it works for both constant and variable arguments.

C++

```cpp
#include <iostream>

constexpr int kFeetPerYard = 3;

constexpr int yards(int feet) { return feet / kFeetPerYard; }

int main() {
    std::cout << "10 feet = " << yards(10) << " yard(s)\n";
    std::cout << "3 feet  = " << yards(3) << " yard(s)\n";
    return 0;
}
```

output

```
10 feet = 3 yard(s)
3 feet  = 1 yard(s)
```

4. `std::cin` splits input on whitespace, and `/` and `%` work only on integers.

C++

```cpp
#include <iostream>

int main() {
    int first = 0;
    int second = 0;
    std::cout << "two integers: ";
    std::cin >> first >> second;

    std::cout << "sum    = " << first + second << '\n';
    std::cout << "diff   = " << first - second << '\n';
    std::cout << "prod   = " << first * second << '\n';
    std::cout << "quot   = " << first / second << '\n';
    std::cout << "rem    = " << first % second << '\n';
    return 0;
}
```

output

```
two integers: sum    = 5
diff   = -1
prod   = 6
quot   = 1
rem    = 2
```

The expected output above was produced with the input `2 3`; the prompt text appears on the same line because the values are typed after it.

2.6 Mini test

1. Which initialisation form refuses a narrowing conversion? A) `int a = 2.5;` B) `int b(2.5);` C) `int c{2.5};` D) `int d = {2.5};`
2. What does `sizeof(int)` return? A) the number of bits the standard guarantees B) the number of bytes an `int` occupies on this toolchain C) the number of decimal digits an `int` can hold D) the alignment of `int`
3. Which declaration makes an object that cannot be modified after initialisation? A) `static int x = 1;` B) `const int x = 1;` C) `mutable int x = 1;` D) `volatile int x = 1;`

4. What is the type of the literal `5`? A) `long long` B) `unsigned int` C) `int` D) `double`
5. True or false: `0u - 1u` has a defined result for unsigned arithmetic.
6. What does `constexpr` promise about a function?

Answer key and explanations

1. C - brace initialisation rejects conversions that lose information, and copy or direct initialisation would silently truncate.
2. B - it is a compile time constant for the target platform, which is why projects do not hard-code 4.
3. B - `const` blocks modification after the object is constructed, and the compiler enforces it.
4. C - a decimal integer literal without a suffix has type `int` if it fits, then `long` or `long long`.
5. True - unsigned arithmetic is defined to wrap modulo 2 to the power of the width, so the result is the maximum value of the type.
6. That the function can be evaluated at compile time when its arguments are constant expressions, which allows its use in `static_assert`, array bounds and `constexpr` variables.